

29

Type Operators and Kinding

In previous chapters, we have often made use of abbreviations like

$$\text{CBool} = \forall X. X \rightarrow X \rightarrow X;$$

and

$$\text{Pair } Y \ Z = \forall X. (Y \rightarrow Z \rightarrow X) \rightarrow X;$$

to make examples easier to read, writing $\lambda x:\text{Pair Nat Bool}. x$, for instance, instead of the more cumbersome $\lambda x:\forall X. (\text{Nat} \rightarrow \text{Bool} \rightarrow X) \rightarrow X. x$.

`CBool` is a simple abbreviation; when we see it in an example, we should just replace it by the right-hand side of its definition. `Pair`, on the other hand, is a *parametric abbreviation*; when we encounter `Pair S T`, we must substitute the actual types `S` and `T` for the parameters `Y` and `Z` in its definition. In other words, abbreviations like `Pair` give us an informal notation for defining functions at the level of type expressions.

We have also used type-level expressions like `Array T` and `Ref T` involving the *type constructors* `Array` and `Ref`. Although these type constructors are built into the language, rather than being defined by the programmer, they are also a form of functions at the level of types. We can view `Ref`, for example, as a function that, for each type `T`, yields the type of reference cells containing an element of `T`.

Our task in this and the next two chapters is to treat these type-level functions, collectively called *type operators*, more formally. In this chapter, we introduce basic mechanisms of abstraction and application at the level of types, along with a precise definition of when two type expressions should be regarded as equivalent and a well-formedness relation, called *kinding*, that prevents us from writing nonsensical type expressions. Chapter 30 goes a

The system introduced in this chapter is the pure simply typed lambda-calculus with type operators, λ_ω (Figure 29-1). The examples also use numbers and booleans (8-2) and universal types (23-1). The associated OCaml implementation is `fullomega`.

step further and treats type operators as *first-class citizens*—i.e., as entities that can be passed as arguments to functions; that chapter introduces the well-known System F_ω , generalizing the quantification over types in System F (Chapter 23) to *higher-order quantification* over type operators. Chapter 31 considers the combination of type operators, higher-order quantification, and subtyping.

29.1 Intuitions

To study functions at the level of types, the first thing we need is some notation for abstraction and application. It is standard practice to use the *same* notations for these as for abstraction and application at the level of terms, indicating abstraction by λ and application by juxtaposition.¹ For example, we write $\lambda X. \{a:X, b:X\}$ for the function that, given a type T , yields the record type $\{a:T, b:T\}$. The application of this function to the argument `Bool` is written $(\lambda X. \{a:X, b:X\}) \text{ Bool}$.

Like ordinary functions, type functions with multiple arguments can be built from one-argument functions by *currying*. For example, the type expression $\lambda Y. \lambda Z. \forall X. (Y \rightarrow Z \rightarrow X) \rightarrow X$ represents a two-argument function—or, strictly speaking, a one-argument function that, when applied to a type S , yields another one-argument function that, when applied to a type T , yields the type $\forall X. (S \rightarrow T \rightarrow X) \rightarrow X$.

We will continue to use informal abbreviations for long type expressions, including type operators. For example, in the remainder of this chapter we will assume we have the abbreviation

$$\text{Pair} = \lambda Y. \lambda Z. \forall X. (Y \rightarrow Z \rightarrow X) \rightarrow X;$$

When we write $\text{Pair } S \ T$ in examples, what we really mean is

$$(\lambda Y. \lambda Z. \forall X. (Y \rightarrow Z \rightarrow X) \rightarrow X) \ S \ T.$$

In other words, we are replacing the informal convention of parametric abbreviation that we have used up to this point with the more elementary informal convention of expanding simple abbreviations to their right-hand sides whenever we see them, plus formal mechanisms for definition and instantiation of type operators. The operations of defining and expanding abbreviations can

1. The one drawback of this notational parsimony is that the terminology for different sorts of expressions can become a little contorted. In particular, the phrase “type abstraction” might now mean an abstraction that expects a type as its argument (i.e., a term like $\lambda X. t$), or it might equally mean an abstraction at the level of types (i.e., a type expression like $\lambda X. \{a:X\}$). In contexts where both are possible, people tend to use “polymorphic function” for the first sense and “type-level abstraction” or “operator abstraction” for the second.

also be treated formally—i.e., we can make them operations in the object language, instead of conventions in the meta-language—but we will not do so here. Interested readers are referred to the literature on type systems with *definitions* or *singleton kinds*; see Severi and Poll (1994), Stone and Harper (2000), Crary (2000), and other works cited there.

Introducing abstraction and application at the level of types gives us the possibility of writing the same type in different ways. For example, if Id is an abbreviation for the type operator $\lambda X.X$, then the expressions

$$\begin{array}{lll} \text{Nat} \rightarrow \text{Bool} & \text{Nat} \rightarrow \text{Id Bool} & \text{Id Nat} \rightarrow \text{Id Bool} \\ \text{Id Nat} \rightarrow \text{Bool} & \text{Id (Nat} \rightarrow \text{Bool)} & \text{Id (Id (Id Nat} \rightarrow \text{Bool))} \end{array}$$

are all names for the same arrow type. To make this intuition precise, we introduce a *definitional equivalence* relation on types, written $S \equiv T$. The most important clause in the definition of this relation

$$(\lambda X :: K_{11}. T_{12}) T_2 \equiv [X \mapsto T_2] T_{12} \quad (\text{Q-APPABS})$$

tells us that a type-level abstraction applied to an argument is equivalent to the body of the abstraction with the argument substituted for the formal parameter. We exploit definitional equivalence in typechecking by a new rule

$$\frac{\Gamma \vdash t : S \quad S \equiv T}{\Gamma \vdash t : T} \quad (\text{T-EQ})$$

precisely capturing the intuition that, if two types are equivalent, then the members of one are all members of the other.

Another new possibility that abstraction and application mechanisms give us is the ability to write meaningless type expressions. For example, applying one proper type to another, as in the type expression (Bool Nat) , makes no more sense than applying true to 6 at the term level. To prevent this sort of nonsense, we introduce a system of *kinds* that classify type expressions according to their arity, just as arrow types tell us about the arities of terms.

Kinds are built from a single atomic kind, written $*$ and pronounced “type,” and a single constructor $\Rightarrow$. They include, for example:

- $*$ the kind of proper types (like Bool and $\text{Bool} \rightarrow \text{Bool}$)
- $* \Rightarrow *$ the kind of type operators (i.e., functions from proper types to proper types)
- $* \Rightarrow * \Rightarrow *$ the kind of functions from proper types to type operators (i.e., two-argument operators)
- $(* \Rightarrow *) \Rightarrow *$ the kind of functions from type operators to proper types

Kinds, then, are “the types of types.” In essence, the system of kinds is a copy of the simply typed lambda-calculus, “one level up.”

In what follows, we use the word *type* for any type-level expression—i.e., both for ordinary types like $\text{Nat} \rightarrow \text{Nat}$ and $\forall X. X \rightarrow X$ and for type operators like $\lambda X. X$. When we want to focus on ordinary types (i.e., the sorts of type expressions that are actually used to classify terms), we call them *proper types*.

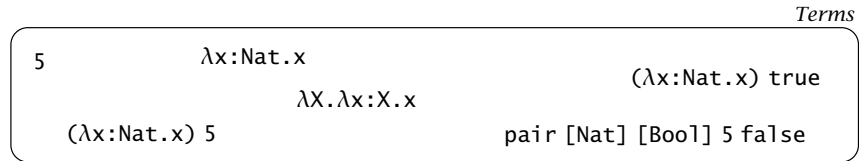
Type expressions with kinds like $(* \Rightarrow *) \Rightarrow *$ are called *higher-order type operators*. Unlike higher-order functions at the term level, which are often extremely useful, higher-order type operators are somewhat esoteric. We will see one class of examples that use them in Chapter 32.

To simplify the problem of checking the well-kindedness of type expressions, we annotate each type-level abstraction with a kind for its bound variable. For example, the official form of the `Pair` operator is:

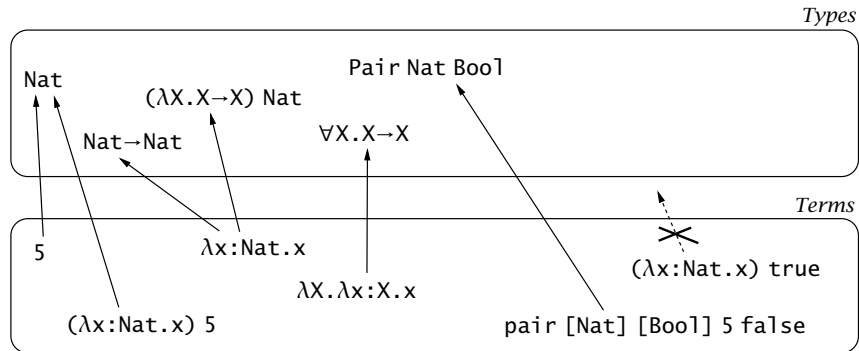
$$\text{Pair} = \lambda A :: *. \lambda B :: *. \forall X. (A \rightarrow B \rightarrow X) \rightarrow X;$$

(Note the doubled colon.) However, since almost all of these annotations will be $*$, we will continue to write $\lambda X. T$ as an abbreviation for $\lambda X :: *. T$.

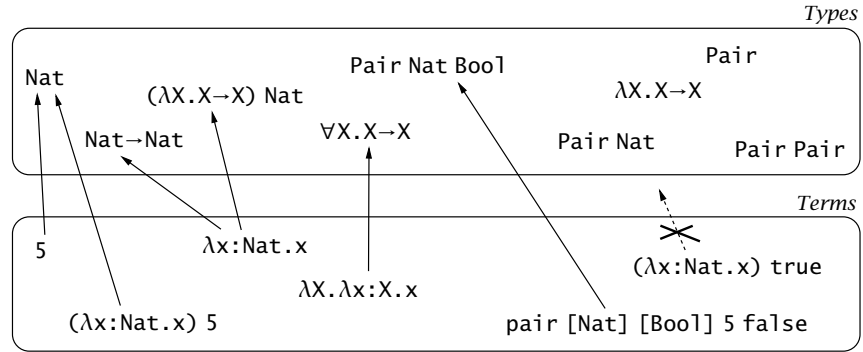
A few pictures may help clarify. The expressions of our language are now divided into three separate classes: terms, types, and kinds. The level of terms contains basic data values (integers, floats), compound data values (records, etc.), value-level abstractions, applications, type abstractions, and type applications.



The level of types contains two sorts of expressions. First, there are proper types like Nat , $\text{Nat} \rightarrow \text{Nat}$, Pair Nat Bool , and $\forall X. X \rightarrow X$, which are inhabited by terms. (Of course, not all terms have a type; for example $(\lambda x : \text{Nat}. x) \text{ true}$ does not.)

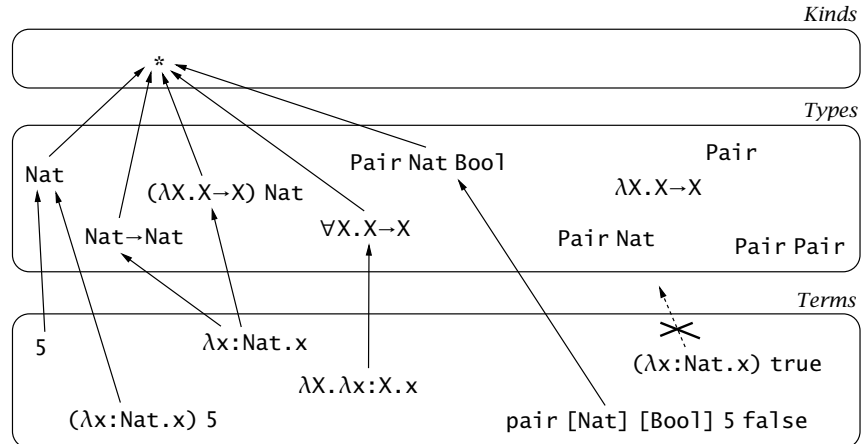


Then there are type operators like `Pair` and $\lambda X.X \rightarrow X$, which do *not* themselves classify terms (it does not make sense to ask “What terms have type $\lambda X.X \rightarrow X$?”), but which can be applied to type arguments to form proper types like $(\lambda X.X \rightarrow X)\text{Nat}$ that do classify terms.

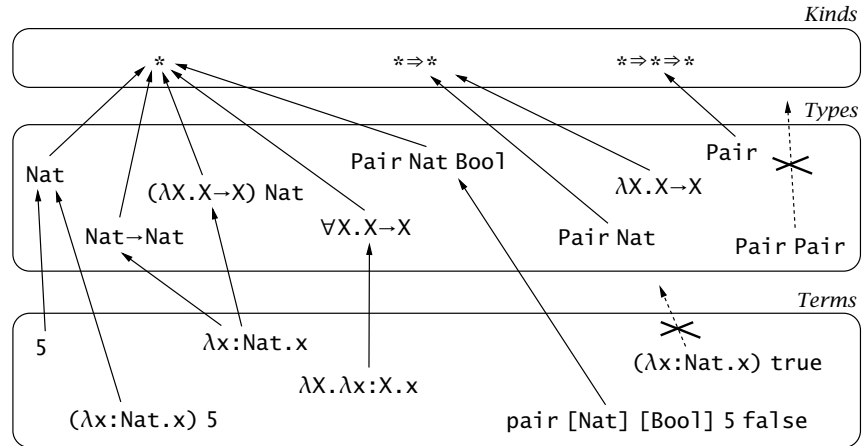


Note that proper types—i.e., type expressions of kind $*$ —may include type operators of higher kinds as subphrases, as in $(\lambda X.X \rightarrow X)\text{Nat}$ or `Pair Nat Bool`, just as term expressions belonging to base types like `Nat` may include lambda-abstractions as subexpressions, as in $(\lambda x:\text{Nat}.x)\ 5$.

Finally, we have the level of kinds. The simplest kind is $*$, which has all proper types as members.



Type operators like $\lambda X.X \rightarrow X$ and `Pair` belong to arrow kinds like $* \Rightarrow *$ and $* \Rightarrow * \Rightarrow *$. Ill-formed type-level expressions, like `Pair Pair`, do not belong to any kind.



- 29.1.1 EXERCISE [★]: What is the difference in meaning between the type-level expressions $\forall X.X \rightarrow X$ and $\lambda X.X \rightarrow X$? □
- 29.1.2 EXERCISE [★]: Why doesn't an arrow type like $\text{Nat} \rightarrow \text{Nat}$ have an arrow kind like $* \Rightarrow *$? □

A natural question at this point is “Why stop at three levels of expressions?” Couldn't we go on to introduce functions from kinds to kinds, application at the level of kinds, etc., add a fourth level to classify kind expressions according to their functionality, and continue on in this way *ad infinitum*? Such systems have been investigated by the *pure type systems* community (Terlouw, 1989; Berardi, 1988; Barendregt, 1991, 1992; Jutting, McKinna, and Pollack, 1994; McKinna and Pollack, 1993; Pollack, 1994). For programming languages, however, three levels have proved sufficient.

Indeed, while type operators can be found, in some form, in essentially all statically typed programming languages, it is relatively rare for language designers to offer programmers even the full power of the present formulation. Some languages (e.g., Java) offer only a few built-in type operators like `Array`, with no facilities for defining new ones. Others bundle type operators together with other language features; in ML, for example, type operators are provided as part of the *datatype* mechanism; we can define *parametric datatypes* like²

2. We're ignoring ML's conventions for capitalization of identifiers for the sake of the example. In OCaml, this definition would really be written

```
type 'a tyop = Tyoptag of ('a → 'a);
```

```
type 'a Tyop = tyoptag of ('a → 'a);
```

which we would write as

```
Tyop = λX. <tyoptag:X→X>;
```

In other words, in ML we can define parametric variants, but not arbitrary parametric types. The benefit of this restriction is that, wherever the type operator `Tyop` appears in a program at the level of types, the corresponding tag `tyoptag` will appear at the level of terms—that is, every place where the typechecker needs to use the definitional equivalence relation to replace a type like `Tyop Nat` by its reduced form `Nat→Nat` will be marked in the program by an explicit occurrence of `tyoptag`. This substantially simplifies the typechecking algorithm.³

The $\Rightarrow$ constructor on kinds is the only one that we have space to discuss here, but a great many others have been studied; indeed, the range of kinding systems for checking and tracking various properties of type expressions rivals the range of type systems for analyzing properties of terms. There are *record kinds* (whose elements are records of types—not to be confused with the types of records; they offer a natural way of defining systems of mutually recursive types), *row kinds* (which describe “rows of fields” that can be used to assemble record types in systems with row variable polymorphism—see page 337), *power kinds* or *power types* (which offer an alternate presentation of subtyping—see Cardelli, 1988a), *singleton kinds* (related to *definitions*—see page 441—and to module systems with *sharing*—see page 465), *dependent kinds* (an analog “one level up” of the dependent types discussed in §30.5), and many more.

29.2 Definitions

Figure 29-1 presents the complete definition of a core lambda-calculus with type operators. At the term level, this calculus includes just the variables, abstraction, and application of the simply typed lambda-calculus (for this reason, it is called the *simply typed lambda-calculus with type operators*). The type level includes the usual arrow types and type variables, plus operator abstraction and application. Quantified types like $\forall X. T$ are omitted from this system; we return to them in detail in Chapter 30.

3. This restriction is similar to ML’s treatment of recursive types, discussed in §20-1. The bundling of recursive types into `datatype` definitions gives the programmer the convenience of equi-recursive types and the typechecker the simplicity of iso-recursive types by hiding the `fold/unfold` annotations in the tagging and case analysis operations associated with variant types.

$\rightarrow \Rightarrow$		Extends λ_{ω} (9-1)	
Syntax		Kinding	$\boxed{\Gamma \vdash T :: K}$
$t ::=$	<i>terms:</i>	$\frac{X :: K \in \Gamma}{\Gamma \vdash X :: K}$	(K-TVAR)
x	<i>variable</i>		
$\lambda x:T.t$	<i>abstraction</i>	$\frac{\Gamma, X :: K_1 \vdash T_2 :: K_2}{\Gamma \vdash \lambda X :: K_1. T_2 :: K_1 \Rightarrow K_2}$	(K-ABS)
$t\ t$	<i>application</i>	$\frac{\Gamma \vdash T_1 :: K_{11} \Rightarrow K_{12} \quad \Gamma \vdash T_2 :: K_{11}}{\Gamma \vdash T_1\ T_2 :: K_{12}}$	(K-APP)
$v ::=$	<i>values:</i>	$\frac{\Gamma \vdash T_1 :: * \quad \Gamma \vdash T_2 :: *}{\Gamma \vdash T_1 \rightarrow T_2 :: *}$	(K-ARROW)
$\lambda x:T.t$	<i>abstraction value</i>		
$T ::=$	<i>types:</i>	$\boxed{S \equiv T}$	Type equivalence
X	<i>type variable</i>	$T \equiv T$	(Q-REFL)
$\lambda X :: K.T$	<i>operator abstraction</i>	$\frac{T \equiv S}{S \equiv T}$	(Q-SYMM)
$T\ T$	<i>operator application</i>	$\frac{S \equiv U \quad U \equiv T}{S \equiv T}$	(Q-TRANS)
$T \rightarrow T$	<i>type of functions</i>	$\frac{S_1 \equiv T_1 \quad S_2 \equiv T_2}{S_1 \rightarrow S_2 \equiv T_1 \rightarrow T_2}$	(Q-ARROW)
$\Gamma ::=$	<i>contexts:</i>	$\frac{S_2 \equiv T_2}{\lambda X :: K_1. S_2 \equiv \lambda X :: K_1. T_2}$	(Q-ABS)
$\emptyset$	<i>empty context</i>	$\frac{S_1 \equiv T_1 \quad S_2 \equiv T_2}{S_1\ S_2 \equiv T_1\ T_2}$	(Q-APP)
$\Gamma, x:T$	<i>term variable binding</i>	$(\lambda X :: K_{11}. T_{12})\ T_2 \equiv [X \mapsto T_2] T_{12}$	(Q-APPABS)
$\Gamma, X :: K$	<i>type variable binding</i>		
$K ::=$	<i>kinds:</i>		
$*$	<i>kind of proper types</i>		
$K \Rightarrow K$	<i>kind of operators</i>		
Evaluation	$\boxed{t \rightarrow t'}$		
$\frac{t_1 \rightarrow t'_1}{t_1\ t_2 \rightarrow t'_1\ t_2}$	(E-APP1)		
$\frac{t_2 \rightarrow t'_2}{v_1\ t_2 \rightarrow v_1\ t'_2}$	(E-APP2)		
$(\lambda x:T_{11}. t_{12})\ v_2 \rightarrow [x \mapsto v_2] t_{12}$	(E-APPABS)		
			<i>continued...</i>

Figure 29-1: Type operators and kinding (λ_{ω})

<i>Typing</i>			
	$\frac{x:T \in \Gamma}{\Gamma \vdash x : T}$	$\boxed{\Gamma \vdash t : T}$	
		(T-VAR)	
	$\frac{\Gamma \vdash T_1 :: * \quad \Gamma, x:T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2}$		(T-ABS)
		$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}}$	(T-APP)
		$\frac{\Gamma \vdash t : S \quad S \equiv T \quad \Gamma \vdash T :: *}{\Gamma \vdash t : T}$	(T-EQ)

Figure 29-1: Type operators and kinding (continued)

The presentation of the system extends the framework of the simply typed lambda-calculus in three ways. First, we add a collection of rules of *kinding*, which specify how type expressions can be combined to yield new type expressions. We write $\Gamma \vdash T :: K$ for “type T has kind K in context Γ .” Note the similarity between these kinding rules and the typing rules of the original simply typed lambda-calculus (Figure 9-1).

Second, whenever a type T appears in a term (as in $\lambda x:T. t$), we must check that T is well formed. This involves adding a new premise to the old T-ABS rule that checks $\Gamma \vdash T :: *$. Note that T must have exactly kind $*$ —i.e., it must be a proper type—since it is being used to describe the values that the term-variable x may range over. The typing rules maintain the invariant that, whenever we can derive a statement $\Gamma \vdash t : T$, the statement $\Gamma \vdash T :: *$ is also derivable (as long as all the types appearing in the context are well kinded). This point is discussed in more detail in §30-3.

Third, we add a collection of rules for the *definitional equivalence* relation between types. We write $S \equiv T$ for “types S and T are definitionally equivalent.” This relation is quite similar to the reduction relation at the level of terms. The effect of definitional equivalence on typing is captured by the new T-EQ rule. The kinding premise (which was elided when we discussed the rule in the previous section) maintains the invariant mentioned above, that “typable terms always have kindable types.” Note the similarity of this rule to the rule of subsumption (T-SUB) in systems with subtyping.

The basic metatheoretic properties of this system require a little work to develop, since the type equivalence relation introduces significant flexibility in the “shapes” of the types assigned to terms. We postpone the development of this theory to Chapter 30.